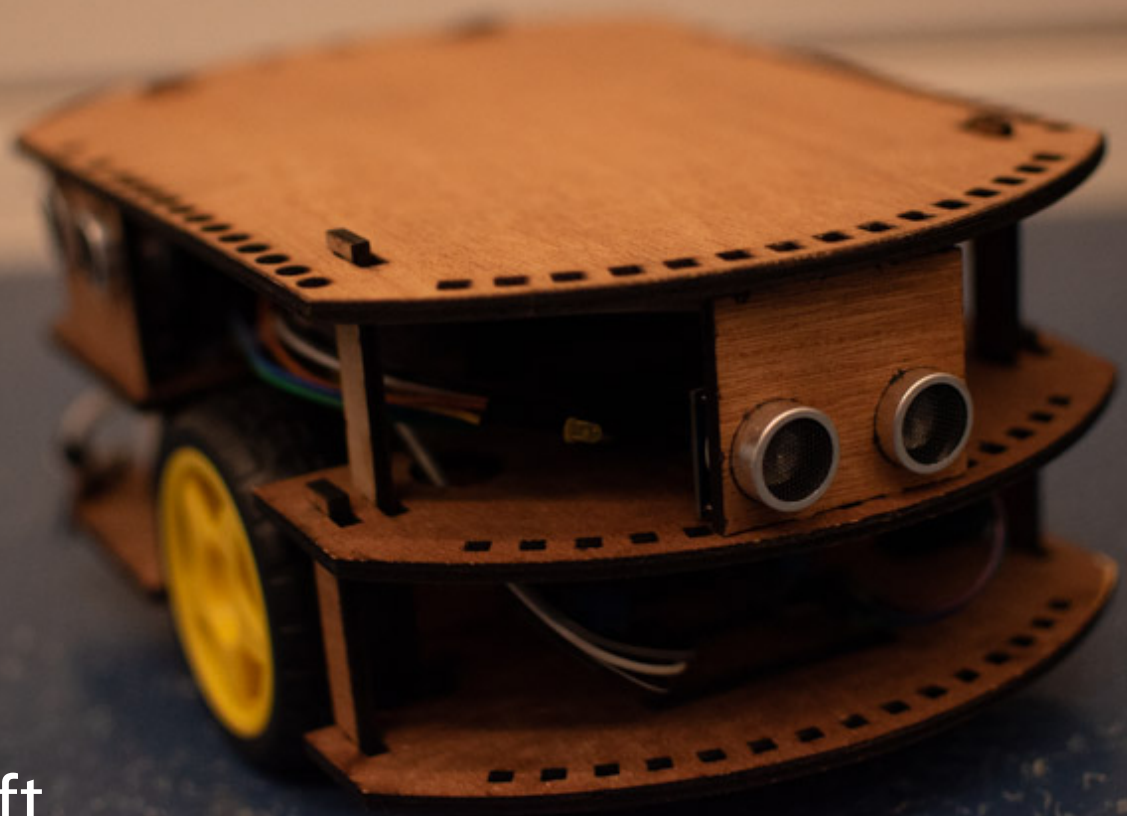


CSE2425 Embedded Software

Lab Manual 2023-2024

Q. Wang
J. Haas
W.J. Baartman
K.G. Langendoen
V.D. van de Beek
W. Narchi
M. Hadjigeorgiou



Preface

When redesigning the Embedded Software course practicals, we realized that the previous robot set up for the assignment was not entirely suitable. In the past years, the students spent lots of time programming on laptops/PCs to detect the lines from the pictures taken by their smartphones. The laptops/PCs then sent the information of the detected lines to the robot through Bluetooth. Based on this information, the robots performed proper actions to follow the lines. There were two motivations for revamping the assignment: 1) Programming and running the codes on laptops/PCs is not a learning objective of this course, and 2) the latency on sending the line detection information from the laptop/PC to the robot is too long. Two years ago, we moved to a setup where students could do regular embedded programming in a microcontroller. Furthermore, following the suggestion given by Prof. Martijn Wisse and Martin Klomp of 3ME, we decided to give students the raw components of the robot. The student assembled the car robot by themselves. This year is the third year that we try this brand-new setup. This year, we have also purchased some new hardware for this assignment and added a screen that could help with the debugging. Based on the validations in the past years, the students should be able to complete the assignments at the final demonstration sessions.

*Qing Wang and Koen Langendoen
Delft, November 2024*

Note to reader

In this manual, there will be a series of text boxes with different icons. Although the information in these boxes might not be required for the lab, they contain useful information.

**Be Aware!**

This type of box warns for frequently occurring mistakes and problems.

**Hint**

This type of box contains a hint that might be useful when solving the exercise.

**Background Information**

This type of box gives more information about the technologies that are used in this exercise.

Contents

1	Introduction	1
1.1	Hardware	1
1.2	Timeline	2
1.3	Deliverables	2
1.4	Passing Criteria	3
2	Setup	4
2.1	Assembling the robot	4
2.2	ST-Link connection	5
2.3	Setting up a virtual machine environment	6
2.4	Setting up a local environment	6
2.4.1	PlatformIO	6
2.4.2	Arduino IDE	8
2.4.3	Connecting to the STM32Programmer	8
3	Assignments	10
3.1	Assignment 1: Variable speed	10
3.2	Assignment 2: Obstacle detection and stopping	10
3.3	Assignment 3: Line follower	11
3.4	Assignment 4: Obstacle avoidance	11
3.5	Assignment 5: Fixed-distance traveling	12
A	A Short Tutorial on STM32 Programming with Registers	13
A.1	The world of registers and peripherals	13
A.1.1	General-Purpose Input/Output (GPIO)	13
A.1.2	Reset and Clock Control (RCC)	14
A.2	Putting it all together	14
A.3	Further improvements	15
B	Changes over course editions	16
	Bibliography	17

1

Introduction

The purpose of this lab assignment is to become familiar with programming in an Embedded Systems environment, which comes with certain limitations. Your main tasks are to program a car robot to detect & avoid obstacles, follow lines, and sprint. You will program the STM32F401CCU6 microcontroller, supported by the Arduino IDE (cf. Section 2 for details). Programming the STM32 can be done using either the STM32duino[6] implementation of the Arduino Platform, ST LL, libopencm3 or the CMSIS[5] library with STM32[8] extensions.

1.1. Hardware

The embedded platform used for the lab is an STM32F401CCU6 [7], which is a microcontroller containing a 32-bit ARM Cortex-M4 core. The full list of hardware that you will use to assemble the robot is as follows:

- 1x STM32F401CCU6 microcontroller [7]
- 2x HC-SR04 ultrasound ranging sensor: To detect obstacles[4]
- 1x L9110 H-Bridge: For power distribution to the motors[3]
- 2x TCRT5000 Infrared reflective sensor: To detect lines [9]¹
- 2x B83609 speed encoder: To measure the rotational speed of the robot [1].
- 1x B10 breadboard special power module 5V/3.3V (*optional*)
- 1x OLED display [2]
- 1x ST-Link
- 1x USB cable
- 1x USB power splitter
- 2x Red LEDs
- 1x Breadboard
- 1x 5000mAh Mi Power bank
- Jumper wires
 - 20x *long* (20 cm) male-to-female
 - 10x *short* (≈ 12 cm) male-to-female
 - 6x male-to-male

¹The infrared sensor's sensitivity can be adjusted by turning the screw-shaped knob with a Phillips head screwdriver; see [the assembly instructions on Brightspace](#) (page 7).

- 2x Motors with gearbox
- 2x Wheels
- 1x Caster wheel
- Wooden frames
- 1x Bag of screws and nuts

1.2. Timeline

This lab consists of several assignments as indicated by table 1.1. We recommend following the timeline outlined below, though this is only a recommendation and not strictly necessary.

Table 1.1: A potential timeline for the second part of the lab.

Week	Assignment
5	Robot component pick up, assembly & testing
6	'Variable speed' & 'Obstacle detection and stopping'
7	'Line follower' & 'Obstacle avoidance'
8	'Fixed-distance traveling'
9	Demo / grading

1.3. Deliverables

In addition to the grading demo, there are a number of deliverables that you must provide in order to pass the lab. These should be handed in on Weblab.

1. **Report:** A short report (recommended length of 3-4 A4 pages, though it may be longer/shorter) detailing your solution for each of the completed sub-assignments. Please specify the logic of your solutions in addition to the framework (Arduino, CMSIS, etc) that you used. More concisely, you should at least have sections for each of the following (assuming you have implemented the corresponding functionality), but you may include additional sections if needed:
 - (a) Description of the obstacle detection algorithm
 - (b) Description of the line detection algorithm
 - (c) Description of the obstacle avoidance algorithm
 - (d) Description of the algorithm on calculating the traveling distance
2. **Code:** The complete source code of your solutions. Each code file should have a header with the format specified in listing 1.1. The source files should be delivered in a ZIP file. If you have used the Arduino IDE, the sketches should be called `a_assignment_b.ino`, where *a* and *b* are your student number and the assignment number respectively.

Listing 1.1: The header for all source files

```

1  /**
2   * Student name: [YOUR NAME]
3   * Student number: [YOUR STUDENT NUMBER]
4   *
5   */

```

1.4. Passing Criteria

The criteria are detailed in table 1.2. For passing the lab you will have to get at least 5 points, for example, by finishing all tasks using the Arduino library. If you perform the tasks with advanced techniques such as timer registers or finish the last two tasks with a higher accuracy, then you will receive a higher grade.

Table 1.2: Criteria

Points Criteria	2.5 pt	2 pt	1.5 pt	1 pt
Variable speed	n/a	n/a	Timer Registers	Basic (analogWrite; <i>can use Arduino library</i>)
Obstacle detection and stopping	ISR Interrupt	Using Arduino Library interrupts	n/a	Basic (busy loop with digitalWrite and -Read; <i>can use Arduino library</i>)
Line follower	n/a	Registers (ADC)	n/a	Basic (analogRead; <i>can use Arduino library</i>)
Obstacle avoidance	n/a	Reorient and follow line after avoidance (<i>use registers</i>)	Reorient and follow line after avoidance (<i>can use Arduino library</i>) OR Obstacle successfully avoided only (<i>use registers</i>)	Obstacle successfully avoided only (<i>can use Arduino library</i>)
Fixed-distance traveling	n/a	Error ≤ 10 cm (<i>use registers</i>)	Error ≤ 10 cm (<i>can use Arduino library</i>) OR 30 cm $\geq$ Error > 10 cm (<i>use registers</i>)	30 cm $\geq$ Error > 10 cm (<i>can use Arduino library</i>)



What we mean with registers

With registers we mean programming using memory constants or low-level libraries like CMSIS/ST LL/libopencm3. Arduino can also be replaced with ST HAL if that is preferred, but this will not give you bonus points.

2

Setup

2.1. Assembling the robot

In order to start working on the assignments, you first need to assemble the robot. **Assembly instructions can be found on the Brightspace page of this course** (<https://brightspace.tudelft.nl/d21/1e/content/595328/viewContent/3331271/View>). **This video might help you greatly with the robot assembly** (<https://youtu.be/ox1F1fDCVAo>).



Be Aware!

In most of the places **the short male-to-female jumper wires are too short**. Please use the long (or short+short, i.e. put two short wires together) male-to-female jumper wires in these places. If the short wires work, then please use them because you only have 20 long male-to-female wires.

If the power module is broken after you use it for some time, you can either 1) contact the TAs to get a new one; or 2) use one of the following three methods to power the H-Bridge/motors (option 3 might be the simplest):

- **Option 1:** Cut the short branch of the power splitter cable and connect it to the H-Bridge directly with a jumper wire, see fig. 2.1a;
- **Option 2:** Cut the short branch of the power splitter cable and connect it to the breadboard, see fig. 2.1b; then connect the H-Bridge to the breadboard as when you have the power module.
- **Option 3:** No need to cut the power splitter cable. You just need to connect the 5V/GND pins of the micro-controllers to the +/- of the breadboard, respectively (see fig. 2.2 for details); and wire other parts as when you have the power module. In this way, you power the H-Bridge/Motors using the micro-controllers. The motors do consume a bit of power which might cause problems in some scenarios.

Figure 2.1: Powering the H-Bridge without the power module.

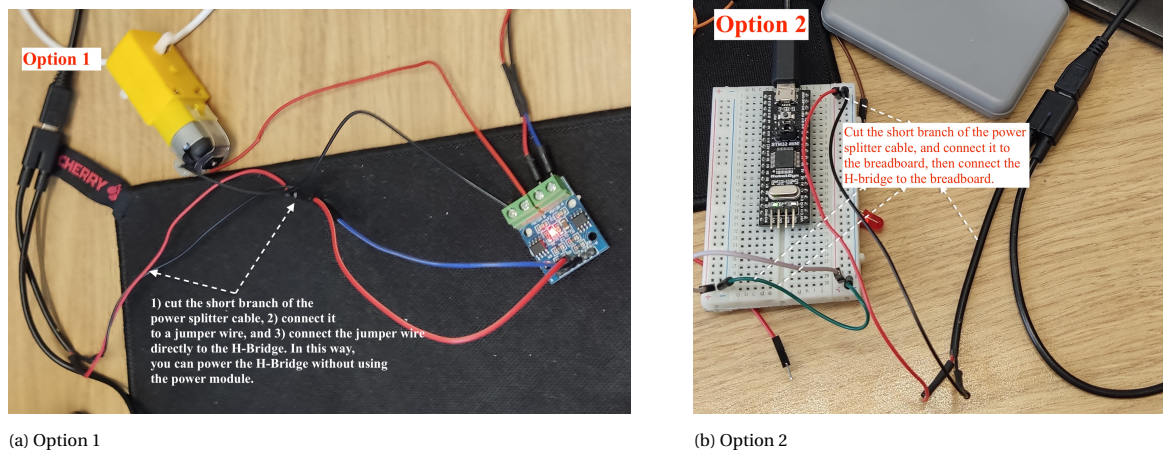
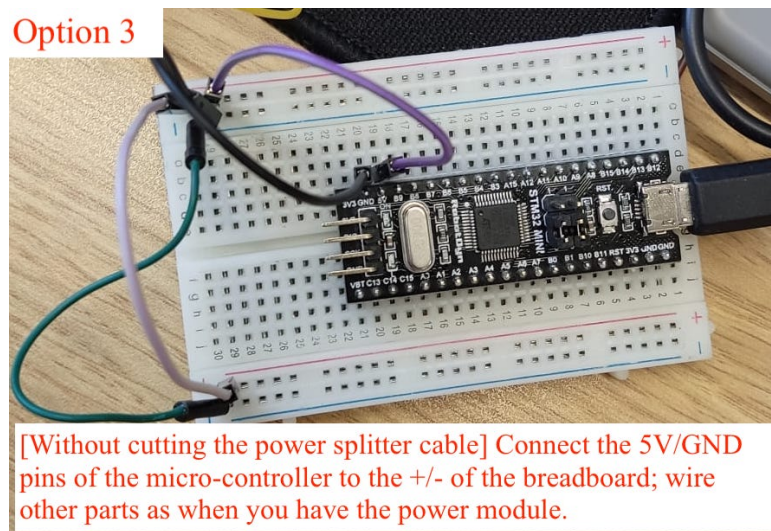


Figure 2.2: Powering the H-Bridge without the power module and without cutting the power splitter cable.

**Overload risk**

The 5V pin on the STM32 can be used to power it. **DO NOT** simultaneously connect the STM32 via USB if you connect the 5V pin to any power source as this may cause a power overload and burn through the controller.

2.2. ST-Link connection

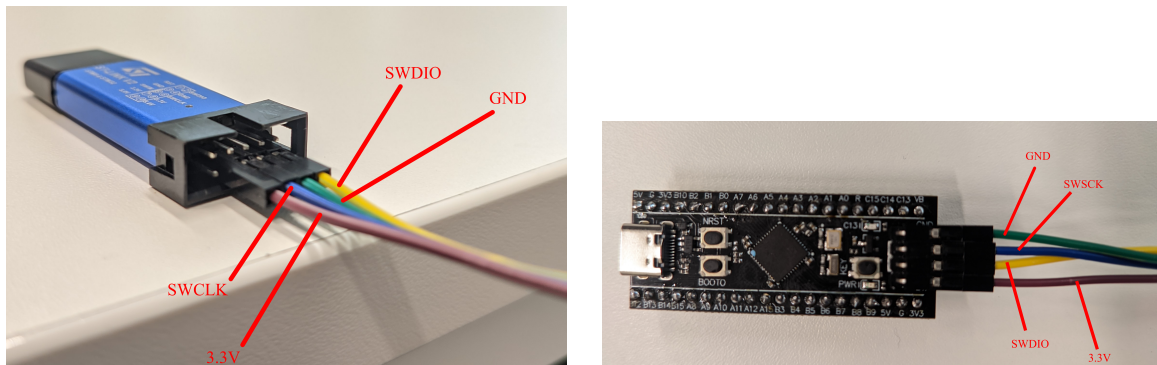
The ST-Link is a debugging and programming dongle made for STM8 and STM32 based microcontrollers. It connects to the given STM32 via the four debug pins on the top of the board and then to a PC via USB.

On the bottom of the STM32, you can see labels for each of its pins. On the casing of the ST-Link, you can see labels for each of its pins. You will need to connect the two sets of pins. You can use the colours of the wires in 2.3 to see an example of a valid connection. Ensure that the following connections are established using female-to-female wires:

- STM32 GND → ST-Link GND
- STM32 SWSCK → ST-Link SWCLK
- STM32 SWDIO → ST-Link SWDIO

- **(Optional)** STM32 3V3 → ST-Link 3.3V

Figure 2.3: Example of how to wire the ST-Link to the STM32



(a) Wiring on the ST-Link

(b) Wiring on the STM32

2.3. Setting up a virtual machine environment

The necessary software to debug, upload and run your code on the STM32 consists of the STM32CubeProgrammer, VS Code with PlatformIO, and the Arduino IDE. These programs have been set up in a **VM (Virtual Machine)** that runs Ubuntu 22.04.01. The VM image can be found on the Brightspace page of this course (<https://brightspace.tudelft.nl/d21/le/content/595328/viewContent/3577264/View>). In order to run it, you need a virtualization application (such as [VirtualBox](#)) and 20 GiB of space.

After installing the VM, you will need to grant it access to the appropriate USB devices. If you are using VirtualBox, this can be done by right-clicking the appropriate VM in VirtualBox Manager, then navigating to Settings > USB, clicking the icon with the green plus then selecting any devices that start with 'STMicroelectronics' and restarting the VM if it is running. You will also need to carry this step out for the serial connection if you wish to have serial output for debugging as detailed in section 2.4.3.

If you are using VirtualBox, it might be useful to set up a shared folder in order to transfer files to and from the VM to your computer¹. You may run into permission issues when you attempt to access the shared folder from your VM².



Skiping the virtual machine

If you choose to avoid setting up the virtual machine, it is possible to install the software yourself. In that case you can follow the instructions in section 2.4. It is advised to only follow these instructions and avoid using the virtual machine if your computer runs Linux, as ST-Link connection errors are much more probable when using Windows.

2.4. Setting up a local environment

In order to start programming for the STM32 in this course, you may also set up a local environment on your own actual computer. This will make use of the ST-Link dongle. It is advised to use PlatformIO (section 2.4.1), but instructions for the Arduino IDE are also provided (section 2.4.2).

2.4.1. PlatformIO

PlatformIO³ is an embedded development environment that is built as a VS Code extension and is supported by all operating systems. It allows for programming STM32-based microcontrollers without having to install any additional dependencies or plugins.

¹<https://carleton.ca/scs/tech-support/troubleshooting-guides/creating-a-shared-folder-in-virtualbox/>

²<https://stackoverflow.com/a/26981786/14247568>

³<https://platformio.org/>

We provide template projects for all frameworks discussed during the hardware lecture, which can be found on the Brightspace page of the course (<https://brightspace.tudelft.nl/d21/1e/content/595328/viewContent/3576897/View>). To create a new project, simply copy all the files from the relevant template folder into a directory on your computer and open this directory in VS Code. PlatformIO should automatically set up a project and download all relevant dependencies.

You must connect your STM32 microcontroller via the ST-Link in order to upload code and debug your program(s). On the bottom left of the VS Code window (see fig. 2.4) you will find buttons to build the project, upload your compiled code, and open a serial monitor to monitor serial output. Hover over the buttons for information text indicating their function.

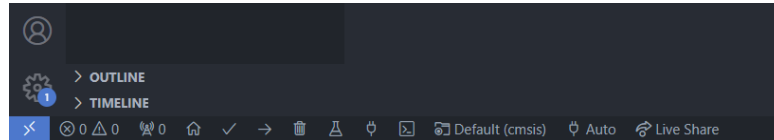


Figure 2.4: PlatformIO bottom bar functionality

You can debug your PlatformIO programs in a GDB-esque environment as well. In order to do this, open up the VS Code command palette (F1 or Ctrl+Shift+P are the relevant keyboard shortcuts) and select PlatformIO: Start Debugging. This will launch a visual debugger that allows you to select breakpoints where execution will be paused and provide you with a number of windows containing useful information. The most relevant are:

- **Variables:** All defined global and local variables as well as their current values.
- **Call Stack:** The sequence of function calls leading up to the point at which execution was paused.
- **Breakpoints:** The breakpoints you have set and their locations.
- **Peripherals:** A list of all memory-mapped I/O registers and their current values. This contains the values of memory locations that control GPIO, timers, interrupts, etc. As such, it is *extremely* useful.

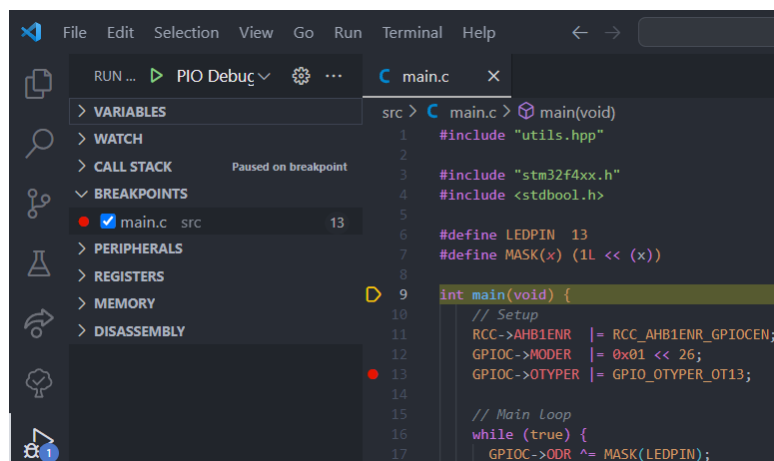


Figure 2.5: PlatformIO debugging view



Example code

Be sure to check out [PlatformIO's STM32 documentation](#) which includes example code for all the libraries that can be used in this course.

2.4.2. Arduino IDE

Programming the STM32 can be done through the Arduino IDE using the ST-Link dongle provided with the robot components. There is official Arduino framework support for STM32 microcontrollers, though this is not set up out-of-the-box. A guide on adding the necessary definitions can be found in the 'Add STM32 boards support to Arduino' section of this guide.⁴

In order to flash your code on to the STM32, you will need to install the STM32CubeProgrammer, which the Arduino IDE makes use of.⁵ Scroll down to 'Get Software' and install version 2.7.0 for your OS.⁶ Ensure that your STM32 can be connected to by opening STM32CubeProgrammer, connecting the ST-Link to your STM32 and your PC, then clicking 'Connect' on the top-right and verifying that board information shows up on the bottom-right.

Lastly, in order to upload code from the Arduino IDE, make sure to set the following options. These can be found under 'Tools' in the top-left corner of the window:

1. Board > STM32 MCU based boards > Generic STM32F4 series
2. Board part number > Blackpill F401CC
3. Upload method > STM32CubeProgrammer (SWD)
4. USB Support > CDC (Generic 'Serial')



Library OLED

Since the OLED screen is used as a debugging tool, you are allowed to use a library to program it. One library that has worked well in the past is [Adafruit SSD1306](#)

2.4.3. Connecting to the STM32Programmer

If STM32CubeProgrammer does not detect your ST-Link after you connect it via USB, it is possible that you need to download some drivers. You can do that by following this tutorial.⁷

Further, it is possible to connect the STM32 directly to your PC via a USB-C cable. This allows you to print to serial, greatly easing the debugging process. You can access serial output via the Arduino IDE by going to Tools > Serial Monitor. The default blink_182.ino sketch provided in the virtual machine image contains an example of how to print to serial.

An example of blinking the built-in LED is given in listing 2.1.

Listing 2.1: An example of blinking the LED

```

1 // the setup function runs once when you press reset or power the board
2 void setup() {
3   // initialize digital pin C13 as an output.
4   // to use pins, prepend 'P' in front of the pin name
5   // as can be found on the board and wiring diagram.
6   pinMode(PC13, OUTPUT);
7 }
8
9 // the loop function runs over and over again forever
10 void loop() {
11   digitalWrite(PC13, HIGH);    // turn the LED on (HIGH is the voltage level)
12   delay(1000);                // wait for a second
13   digitalWrite(PC13, LOW);     // turn the LED off by making the voltage LOW
14   delay(1000);                // wait for a second
15 }
```

⁴<https://github.com/stm32duino/wiki/wiki/Getting-Started#add-stm32-boards-support-to-arduino>

⁵<https://www.st.com/en/development-tools/stm32cubeprog.html>

⁶The odd choice of version is due to support for the components used in the lab being spotty on newer versions.

⁷<https://freeelectron.ro/installing-st-link-v2-to-flash-stm32-targets-on-linux>

**Compiling and Running C**

You **CANNOT** upload code on your microprocessor and have your ST-Link connected through the STM32CubeProgrammer at the same time. You need to disconnect your ST-Link from the STM32CubeProgrammer *before* you can upload your code through the Arduino IDE.

**Connection difficulties**

It is possible that a functional connection cannot be established. This will be indicated by erroneous information such as a non numeric serial number in the 'ST-Link configuration' panel within STM32CubeProgrammer. In that case, your ST-Link may be defective or incompatible with your computer's software configuration; contact the TAs in order to resolve the situation.

3

Assignments

You will need to use STM32 registers in the assignments. Please refer to **Appendix A** for a short tutorial on how to blink an LED with registers in STM32.

3.1. Assignment 1: Variable speed

In this assignment, the program you will write needs to control the motors using PWM signals and timers. The program should perform the following actions in sequence:

- Starts the robot and makes it run (*note: you will need to set the duty cycle of the PWM signal to above a certain threshold (say, above 50% in our test) to make the robot move*).
- After three to four seconds of keeping the robot running at a constant speed, increase the duty cycle of the PWM signals to speed up the robot.
- After another three to four seconds, slow down the robot gradually and finally stop the robot.

In this assignment, you can either 1) place the robot on the floor, or 2) hold it with your hands in the air. In the latter case, the robot as a whole will not move, but the wheels will move. Both of these two methods are accepted in the final demonstration of this assignment.



About PWM

Some timers of the STM32 also control PWM output. Check the documentation and pin mapping to make sure that you don't introduce conflicts in your sketch.



Motor Speeds

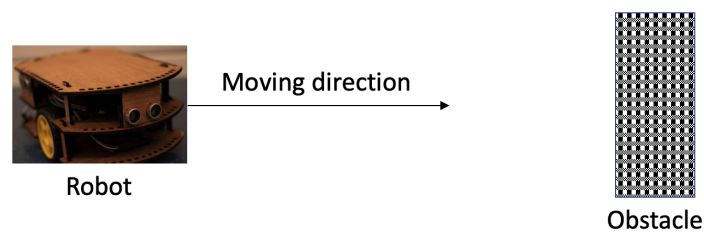
Since the course is using DC motors, it is not possible to get an exact speed. It is therefore likely that your implementation will have a slight drift. This will be accepted in the final demonstration.

3.2. Assignment 2: Obstacle detection and stopping

The robot is equipped with two HC-SR04 ultrasound ranging sensors: one placed in front, and the other placed on the right side of the robot. In this assignment, you will use the **front** ultrasound ranging sensor to detect an obstacle in front of the robot. You can use ISR interrupts (2.5 pts), Arduino library interrupts (2 pts), or an Arduino library busy loop (1 pt). Please refer to table 1.2 for the detailed passing criteria.

The obstacle can be anything you find at home (such as a book or a box). For the final assessment, the obstacle used will be the IKEA box that you received the robot hardware in. You can set up the testing environment as in fig. 3.1.

Figure 3.1: Setup for assignment 2



Your program should perform the following activities in sequence:

- Start the robot and make it run.
- Detect if there is an obstacle ahead using the front ultrasound ranging sensor.
- Stop the robot when it comes close to the obstacle (when the inter-robot-obstacle distance is less than 15 cm).

3.3. Assignment 3: Line follower

The robot has two TCRT5000 Infrared reflective sensors (including two infrared LEDs) attached to the bottom. The LEDs shine infrared light on the floor and the reflected light is detected by the sensors. Based on the reflected light received at the two sensors, a line follower program can be made.

For this assignment, the robot should follow a (black) line on the floor, including any curves. You should DIY (do-it-yourself) the lines, such as using printed papers with (black) lines or with black tape. For the assessment, black electrical tape will be used



Line width

The width of the lines used during the final assessment will be approximately 20mm. You are expected to follow a 'keep in' approach for following lines; the width of the lines is smaller than the distance between the two IR sensors on the bottom of the robot. As such, you should try to keep the line between the two IR sensors.

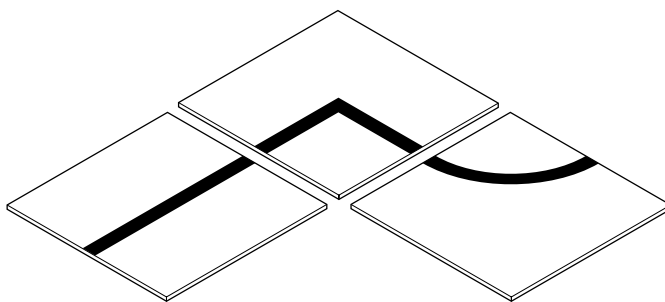


Figure 3.2: Setup for assignment 3: (left) example pieces of track that the robot should be able to navigate; (right) black tape that you might use to build the line on the floor

3.4. Assignment 4: Obstacle avoidance

In this assignment, you will program the robot to maneuver around an obstacle on the path. For the final assessment, the obstacle used will be the IKEA box that you received the robot hardware in (similarly to assignment 2). This obstacle will always be in the middle of the path, i.e. there will be a black line segment before the object and a black line segment after the object.

More concretely, a program to solve this assignment should perform the following:

- Start the robot and make it run by following a straight line that you build (use the same method as assignment 3).
- Detect if there is an obstacle ahead using the front ultrasound ranging sensor
- Maneuver around the obstacle and reach the line on the other side of the obstacle
- **Optional:** Reorient the robot and continue to follow the line (bonus points, see table 1.2)

You will gain additional points for not utilising the Arduino library and doing manual register programming instead.

**Side sensor**

The robot components include a second HC-SR04 ultrasound sensor that is mounted on the side. Though not strictly necessary, it might be useful to utilise this sensor in order to determine when you can turn the robot in order to re-align with the path.

3.5. Assignment 5: Fixed-distance traveling

The robot has two B83609 speed encoders that can be used to measure the robot's rotational speed. In this assignment, you should program the robot to run for a given distance, e.g. 1.5 meters. Your program should perform the following activities in sequence:

- Start the robot and make it run by following a straight line that you build (use the same method as Assignment 3).
- Stop the robot at the place where it is $X.Y$ meters away from the starting position.

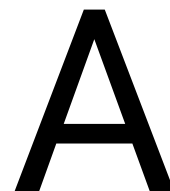
You will gain additional points for not utilising the Arduino library and doing manual register programming instead. Further, you will gain additional points if your fixed distance travelling has an error of < 10 cm (see table 1.2).

**Distance**

Your program should be flexible enough to make the robot run for any given distance. In the final demonstration on campus, you will be given a distance from the range between 1.0 and 2.5 meters.

**Noise**

The B83609 speed encoders are not perfect sensors and so are prone to noise that can affect your algorithm. It might be useful to look into **debouncing** to counteract this.



A Short Tutorial on STM32 Programming with Registers

When using the Arduino IDE, it is customary to use the provided library to program your microcontroller. The IDE implicitly puts `#include <Arduino.h>` at the top of your code to load definitions for functions such as `pinMode` or `digitalWrite`, which are used in the Blink example sketch, for example.

This library makes our job easier, but also keeps us from learning how the STM32 really makes an LED blink. This appendix aims to explain how to blink an LED, with the hopes of setting you on the path of figuring out how to program the STM32 to do more complex tasks.



Your new best friend

For the following sections, keeping the STM32F401 reference manual close is important! This manual can be found [here](#).

A.1. The world of registers and peripherals

The Arduino library we are using is built on top of two the libraries: **STM32 MCU HAL + LL** and **CMSIS**. STM32 MCU HAL + LL is the official software development package which aims to provide a consistent SDK for programming ST microcontrollers. It consists of three parts: a high level hardware abstraction layer (HAL) which standardizes API calls across platforms, a low level API (LL) which consists of low-level & lightweight C calls that correspond to the underlying hardware and CMSIS-compatible definitions of MCU-specific peripherals like the GPIO or ADC.

CMSIS is an ARM-defined standard interface to program the registers and is mainly just definitions on the locations in memory that correspond to different devices. This library is typically the standard definitions for the ARM processor (M4 in our case) and an additional set for the devices added by the chip designer.

These functions read and write data from so-called **registers**, which are special regions in memory that control the processor and its peripherals. Peripherals are internal devices that assist the processor in its tasks, such as keeping track of time, internet connectivity and CRC calculation. In order to find out how to use these registers, we will make use of the [reference manual](#) provided to us by STMicroelectronics.

A.1.1. General-Purpose Input/Output (GPIO)

One such peripheral is the GPIO controller; in truth, these are actually multiple peripherals, each called GPIOX, with X denoting a letter, each mapping to max 16 pins. These mappings are clearly visible on the board: For example, PB4 maps to GPIO Port B (GPIOB), pin 4. Each one of these ports has multiple registers that control its behaviour.

So how do we use this peripheral to control the pins in such a way to make an LED blink? Looking at the manual, one might spot ODR, which stands for Output Data Register. Writing to this should turn pins off and/or on, right?



Accessing registers in CMSIS

The defines for registers are contained in a series of header files belonging to CMSIS. Peripherals are represented by structs, accessed with NAME, with its members being the registers.

You decide to toggle PC13, which connects to the on-board LED. This pin is turned on by setting the 13th bit of the ODR for GPIOC to high. You try the following line:

```
#define MASK(x) (1UL << (x))
#define LEDPIN 13

GPIOC->ODR |= _MASK(LEDPIN);
```

Only to find out the LED doesn't turn on! What is going on? Reading the manual some more, you stumble upon MODER. This is the register that controls the mode of the GPIO pin. A mode is the way that the pin behaves and defines what it will listen to. For the STM32F4 there are four modes defined: input (00), general purpose output (01), alternative function (10) and analog mode (11). As you may notice, we need two bits to explain what mode we want to set it to.

Another important value is the OTYPER register which defines how the output of the port will behave. This can either be push-pull or open-drain. If you remember your Digital Systems or Computer Organization correctly, you will know that this defines whether the transistor connects only to ground or to both power & ground.

We can check the manual, but our header files luckily have proper definitions for the modes we want (found in [stm32f401xc.h](#)). Since we want to blink a LED we want to set the pin into output mode. Since every pin starts as *floating* mode, it is only required that we set the last value to 1. This can be done by ORing the 0th value of the 13th entry in the MODER register of the GPIOC device. This sounds very complicated, but can be easily simplified using the predefined definition GPIO_MODER_MODE13_0.

```
// [DEVICE]->[REG] |= [DEVICE]_[REG]_[VAL][NUM]_[BIT]
// GPIOC->MODER |= 0b01 << (13 * 2)
GPIOC->MODER |= GPIO_MODER_MODE13_0;
```

For the OTYPER register, it is preferred to put it in *push-pull* mode. However we notice that this is the reset, or default, state in our microcontroller. Therefore we can safely keep it at its default value.

It *still* doesn't work? Having lost motivation, you decide to give up. Some light reading will calm you down a bit, so you decide to read a different section of the reference manual, when it suddenly hits you: Peripherals are not enabled by default! This is in order to lower power usage. There exists another peripheral that controls all peripherals: RCC.

A.1.2. Reset and Clock Control (RCC)

The task of the RCC peripheral is to not only enable and reset other peripherals, but also distribute clock signals around to those peripherals. We won't need to change a lot of those settings, so our interaction with RCC will be brief.

RCC is enabled by default, so we don't need to jump through hoops to turn this on.

There are multiple registers to enable peripherals, so we need to find the proper register for this. Checking the manual (or the definitions found in [stm32f401xc.h](#)), we find GPIOC can be turned on with AHB1ENR.

Once again, we add the following to `setup()`, we don't need to clear anything this time:

```
RCC->AHB1ENR |= RCC_AHB1ENR_IOPCEN;
```

You upload the code and... finally, the LED turns on! But it's not blinking yet, so let's wrap everything up and make it blink.

A.2. Putting it all together

In this short tutorial, we've looked at how to use the GPIO peripherals to control the voltage on a pin, as well as using RCC to enable a peripheral. This provides a basic understanding of how the STM32 works.

Finally, let's make the LED blink rather than simply shining continuously. If you may recall from our hardware lecture there are few ways to achieve this. A good method is by using the TIMEx devices to interrupt the microcontrollers. These exceptions can then be controlled either by the generic ARM *Nested Vector Interrupt*

Controller peripheral or the STM32 specific *EXTI*. Another, less efficient option would be using the *Real-Time Clock* that is included. However to save one energy cost, we decide to use our STM32 as a cheap space heater and running a busy loop. To do so, we simply do a loop that counts down and executes a *noop* instruction. After this loop is done, we *xor* the value in the register at position 13 to turn it off and on.

Our program won't differ much from the blink example using just the Arduino library. It still follows the traditional setup & loop architecture. However we pick and choose exactly which registers we write to rather than using complicated functions that may write to many registers at the same time. This will save quite a bit of memory & time and will make your program more predictable. However keep in mind that you do need to spend a lot more time writing your program and that the speed you gained comes at the cost of readability. So always keep this trade-off in mind as you pick what parts you write in Arduino, higher level C library or registers!

We finally arrive at the following:

```

1 #define LEDPIN 13
2 #define MASK (1L << (x))
3
4 void ms_delay(int ms)
5 {
6     while (ms-- > 0) {
7         volatile int x=500;
8         while (x-- > 0)
9             __asm("nop");
10    }
11 }
12 void setup() {
13     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;
14     GPIOC->MODER |= 0b01 << 26;
15     GPIOC->OTYPER |= GPIO_OTYPER_OT13;
16 }
17
18 void loop() {
19     // put your main code here, to run repeatedly:
20     GPIOC->ODR ^= MASK(LEDPIN);
21     ms_delay(300);
22 }

```

Look ma, no Arduino registers! Are you proud of me now?! I have made a small LED blink!

A.3. Further improvements

Now you know how to control registers, you can control everything the STM32 has to offer. The first step would be to understand how to create ISRs. The places where these are defined are found in part of the [vector table](#). Because ISRs are defined as weak, they can be overwritten. For example, to override the SysTick ISR:

```

1 extern "C" void SysTick_Handler(void) {
2     // Your code here
3 }

```

However please be aware that this will *not* always work with STM32Duino since it already overrides some handlers. You may, when working with registers, opt to use *platformio* instead to avoid conflicts with the Arduino library implementation.

This tutorial should've given you the tools required to go out and do your own research. Further inspection of the library is useful to understand the many things the STM32 is capable of. Additionally, many tutorials can be found, but keep in mind register definitions may differ between libraries and processors. A few interesting options are:

- [Embedded Systems Fundamentals with Arm Cortex-M based Microcontrollers: A Practical Approach Nucleo-F091RC Edition](#)
- [Efficient Embedded Systems Design Education Kit for STM32F401](#)
- [Fundamentals of System-on-Chip Design on Arm Cortex-M Microcontrollers](#)
- [Making Embedded Systems](#)

B

Changes over course editions

Edition 2021-2022

- Additional ways to power the STM32.

Edition 2022-2023

- Move to more modern STM32F401 from the STM32F103.
- Addition of small OLED display.
- STLink for reprogramming Z0eff.
- Replace assignment 4 which involved recognizing a robot in front with another assignment where Z0eff navigates around a box.
- Use of the STM32duino over the ArduinoSTM32 Core used previously.
- STM32LL as Arduino backend rather than LibMaple.
- Registers are clarified as being programmed in ARM CMSIS.

Edition 2023-2024

- Clarify final deliverables with dedicated report section
- Provide specifications of line and objects used for the assignments
- Move to PlatformIO as first-class development environment and add usage instructions

Bibliography

- [1] Double speed measuring module with photoelectric encoders model: Hc-020k. URL <https://mrelectrobot.com/wp-content/uploads/2021/09/HC-020K.pdf>.
- [2] Oled display, Dec 2022. URL <https://www.aliexpress.com/item/32957309383.html>.
- [3] Elecrow. L9110 Motor control driver chip. page 2, 2018. URL <https://www.elecrow.com/download/datasheet-l9110.pdf>.
- [4] Indoware. Ultrasonic Ranging Module HC - SR04. *Datasheet*, pages 1–4, 2013. URL <https://www.electroschematics.com/wp-content/uploads/2013/07/HCSR04-datasheet-version-1.pdf>.
- [5] Arm Ltd. Processor support, Nov 2022. URL https://arm-software.github.io/CMSIS_5/Core/html/index.html.
- [6] Frederic Pillon. Stm32duino/arduino_core_stm32: Stm32 core support for arduino, Nov 2022. URL https://github.com/stm32duino/Arduino_Core_STM32.
- [7] STMicroelectronics. STM32F401CC High-performance access line, Arm Cortex-M4 core with DSP and FPU, 256 Kbytes of Flash memory, 84 MHz CPU, ART Accelerator . (August):1–117, 2015. URL <https://www.st.com/en/microcontrollers-microprocessors/stm32f401cc.html#documentation>.
- [8] MCD Application Team. Cmsis_device_f4/stm32f401xc.h at master · stmicroelectronics/cmsis_device_f4, Nov 2022. URL https://github.com/STMicroelectronics/cmsis_device_f4/blob/master/Include/stm32f401xc.h.
- [9] Vishay. Application of optical reflex sensors - mouser electronics. URL https://www.mouser.com/datasheet/2/427/VISH_S_A0003153719_1-2568779.pdf.